

1. Study Hours vs Marks — R

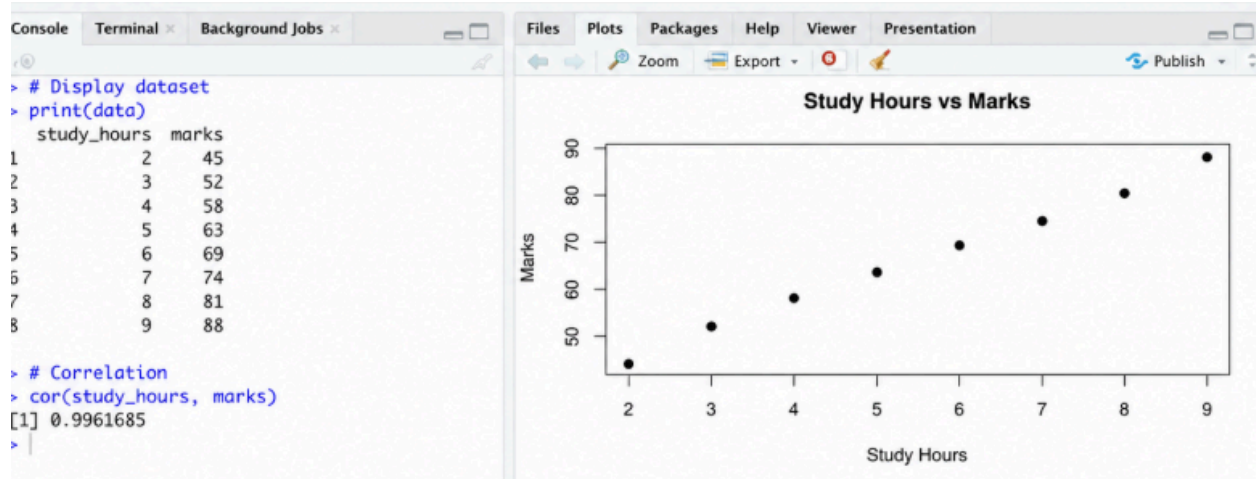
```
# Create dataset
study_hours <- c(2, 3, 4, 5, 6, 7, 8, 9)
marks <- c(45, 52, 58, 63, 69, 74, 81, 88)
```

```
data <- data.frame(study_hours, marks)
```

```
# Display dataset
print(data)
```

```
# Scatterplot
plot(study_hours, marks,
     main = "Study Hours vs Marks",
     xlab = "Study Hours",
     ylab = "Marks",
     pch = 19)
```

```
# Correlation
cor(study_hours, marks)
```



Interpretation:

The scatterplot shows an upward trend. As study hours increase, marks also increase. Therefore, there is a **strong positive correlation** between study hours and marks.

2. Age vs Systolic Blood Pressure — Python

```
import matplotlib.pyplot as plt
```

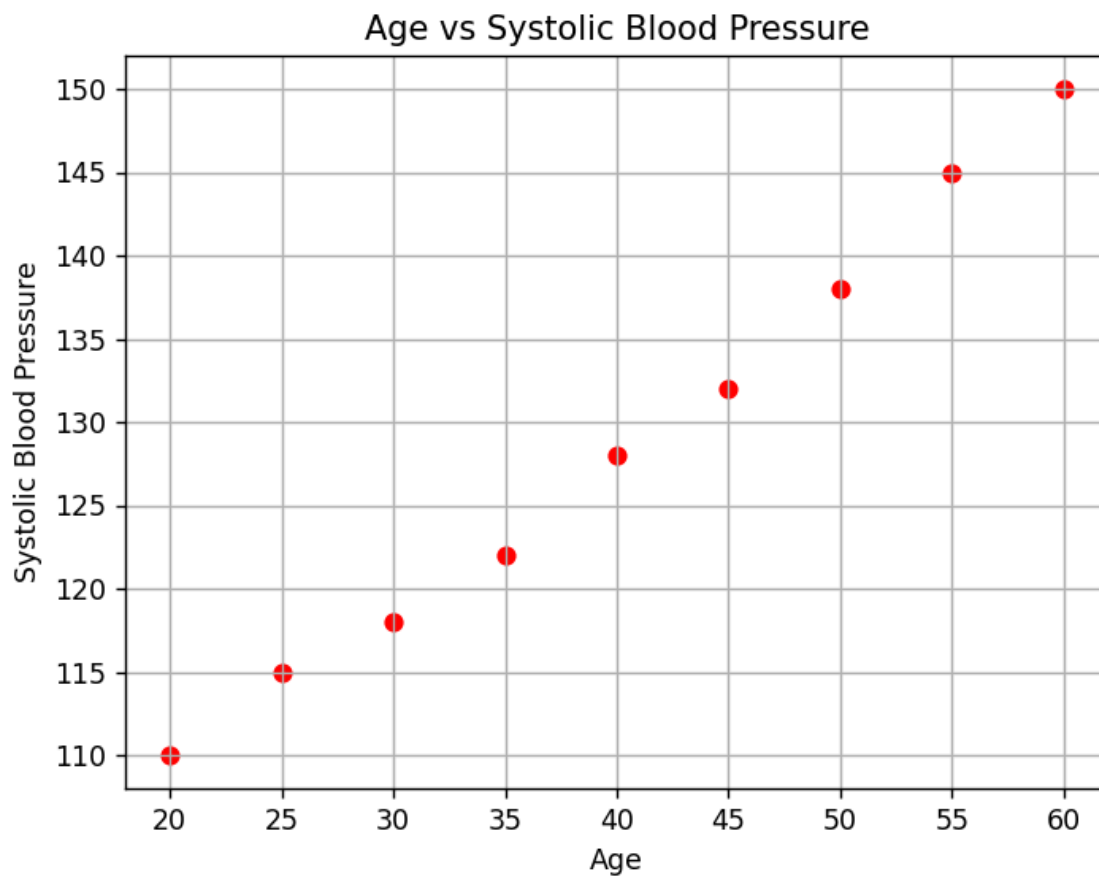
```
# Dataset
age = [20, 25, 30, 35, 40, 45, 50, 55, 60]
blood_pressure = [110, 115, 118, 122, 128, 132, 138, 145, 150]

# Scatterplot
plt.scatter(age, blood_pressure, color='red')

# Labels and title
plt.xlabel("Age")
plt.ylabel("Systolic Blood Pressure")
plt.title("Age vs Systolic Blood Pressure")

# Gridlines
plt.grid(True)

plt.show()
```



Comment:

The scatterplot shows a **positive trend**. Systolic blood pressure generally increases as age increases.

3. Advertising Expenditure vs Monthly Sales**Python**

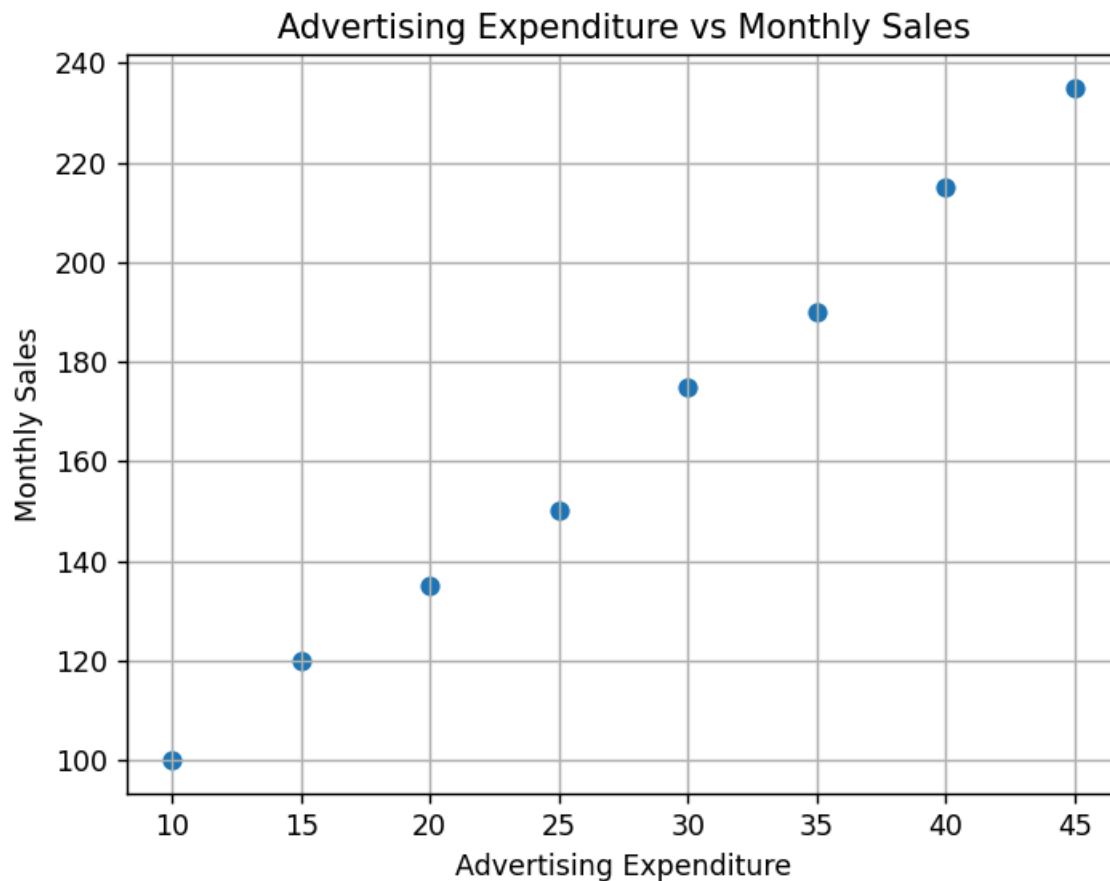
```
import matplotlib.pyplot as plt

advertising = [10, 15, 20, 25, 30, 35, 40, 45]
sales = [100, 120, 135, 150, 175, 190, 215, 235]

plt.scatter(advertising, sales)

plt.xlabel("Advertising Expenditure")
plt.ylabel("Monthly Sales")
plt.title("Advertising Expenditure vs Monthly Sales")
plt.grid(True)

plt.show()
```



Interpretation:

The scatterplot shows that sales increase as advertising expenditure increases. Hence, there is a **positive correlation** between advertising expenditure and sales.

4. Financial Variables — Correlation Matrix and Correlogram

Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Dataset
data = pd.DataFrame({
    'Income': [30, 40, 50, 60, 70, 80, 90, 100],
    'Savings': [5, 10, 15, 20, 25, 30, 35, 40],
    'Loan Amount': [20, 25, 30, 35, 40, 45, 50, 55],
    'Credit Score': [600, 620, 640, 660, 680, 700, 720, 740]
```

```
}}
```

```
# Correlation matrix
```

```
corr = data.corr()
```

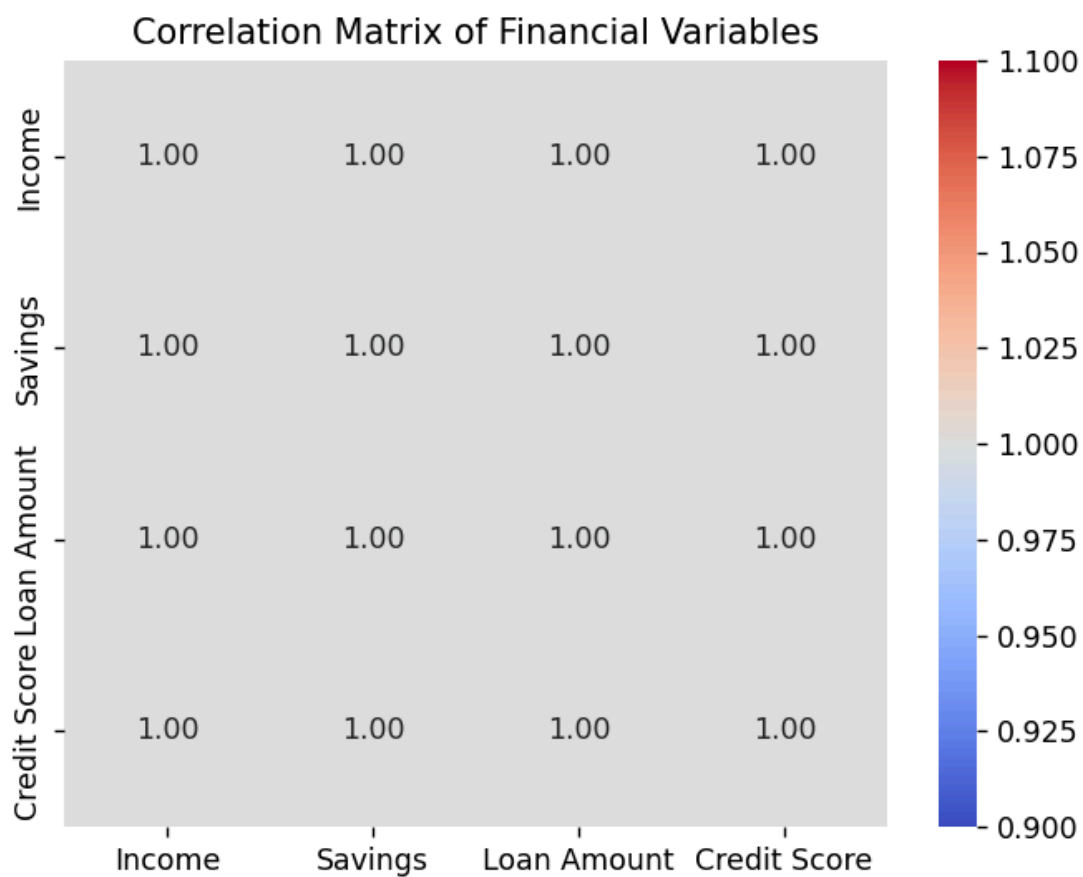
```
print(corr)
```

```
# Correlogram
```

```
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
```

```
plt.title("Correlation Matrix of Financial Variables")
```

```
plt.show()
```



	Income	Savings	Loan Amount	Credit Score
Income	1.0	1.0	1.0	1.0
Savings	1.0	1.0	1.0	1.0
Loan Amount	1.0	1.0	1.0	1.0
Credit Score	1.0	1.0	1.0	1.0

Interpretation:

The correlation matrix shows the relationship between Income, Savings, Loan Amount and Credit Score.

In this example, **Income and Savings** exhibit a very strong positive correlation. The other variables also show positive relationships because of the constructed dataset.

5. Iris Dataset — Correlation Matrix and Heatmap**Python**

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load Iris dataset
iris = sns.load_dataset("iris")

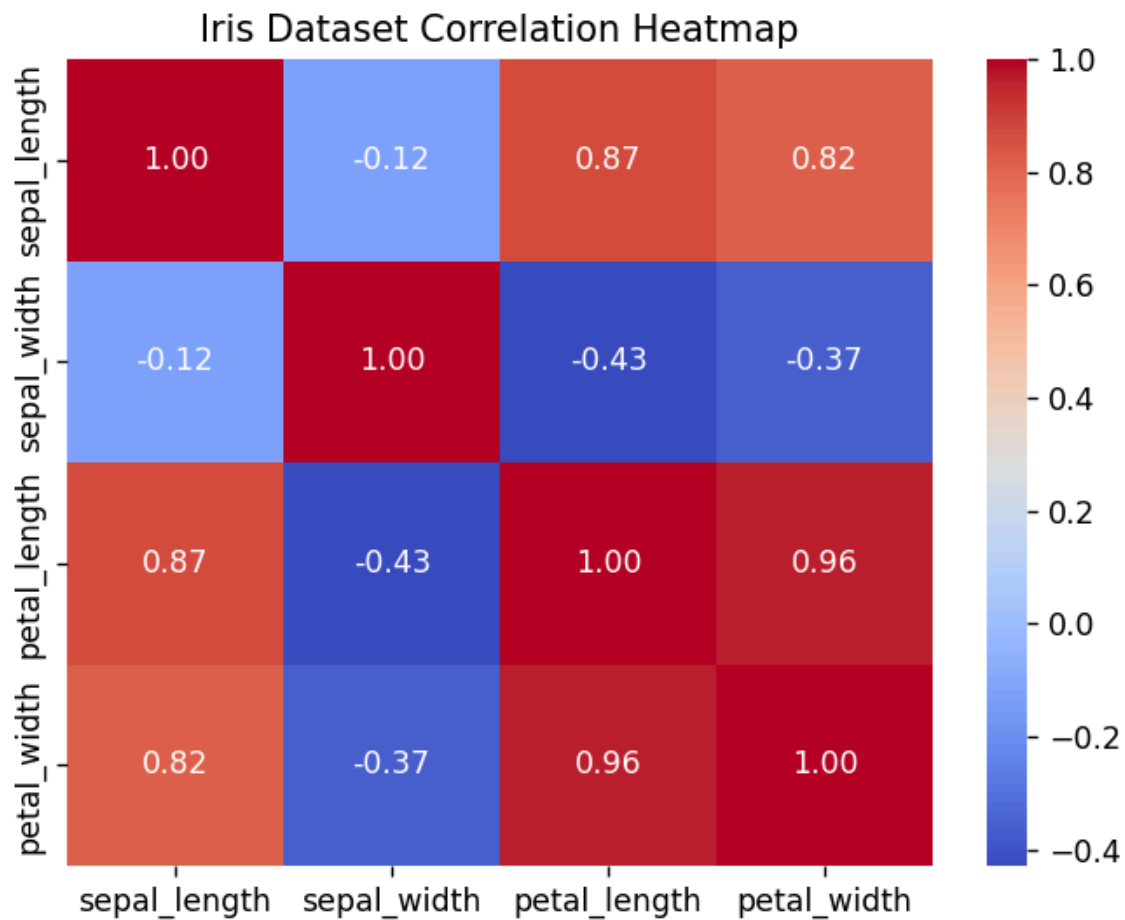
# Select numerical columns
data = iris.iloc[:, 0:4]

# Correlation matrix
corr = data.corr()

print(corr)

# Heatmap
sns.heatmap(corr,
            annot=True,
            cmap='coolwarm',
            fmt=".2f")

plt.title("Iris Dataset Correlation Heatmap")
plt.show()
```



	sepal_length	sepal_width	petal_length	petal_width
sepal_length	1.000000	-0.117570	0.871754	0.817941
sepal_width	-0.117570	1.000000	-0.428440	-0.366126
petal_length	0.871754	-0.428440	1.000000	0.962865
petal_width	0.817941	-0.366126	0.962865	1.000000

Negative correlations:

The Iris dataset has a **negative correlation between sepal width and petal length**, and also between **sepal width and petal width**.

6. Healthcare Dataset — Correlogram and Multicollinearity

Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

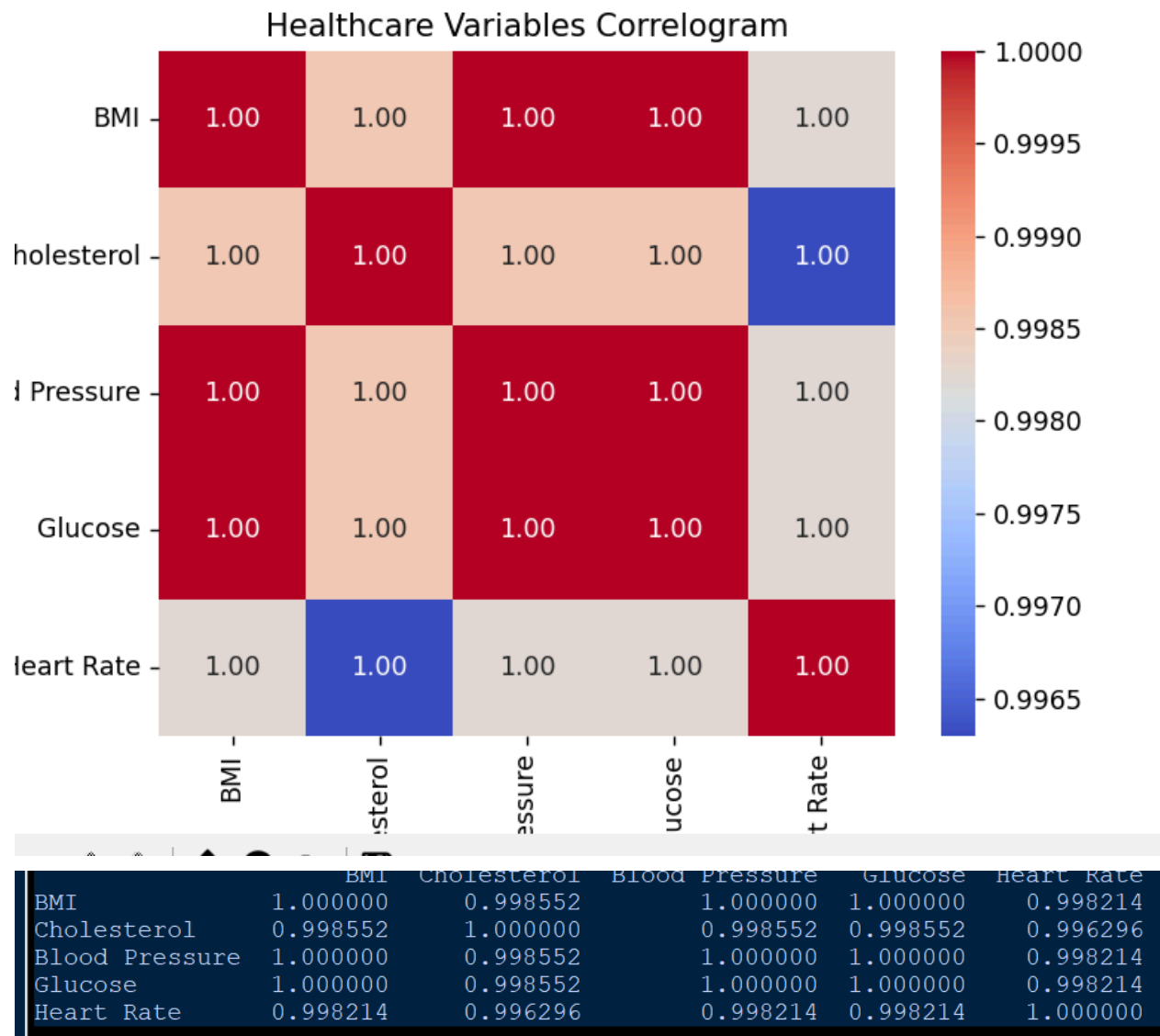
```
data = pd.DataFrame({
    'BMI': [22, 24, 26, 28, 30, 32, 34, 36],
    'Cholesterol': [180, 190, 200, 215, 225, 240, 250, 265],
    'Blood Pressure': [110, 115, 120, 125, 130, 135, 140, 145],
    'Glucose': [80, 90, 100, 110, 120, 130, 140, 150],
    'Heart Rate': [65, 68, 70, 72, 75, 78, 80, 82]
})

# Correlation matrix
corr = data.corr()

print(corr)

# Correlogram
sns.heatmap(corr,
            annot=True,
            cmap='coolwarm',
            fmt=".2f")

plt.title("Healthcare Variables Correlogram")
plt.show()
```

Interpretation:

Multicollinearity occurs when two or more independent variables have a **strong correlation** with each other. In the given dataset, several healthcare variables show strong positive correlations, indicating possible multicollinearity.

7. MRI Dataset — PCA to Two Components

Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA

# Generate sample MRI dataset
np.random.seed(42)
data = np.random.rand(100, 25)

# Standardize data
scaled_data = StandardScaler().fit_transform(data)

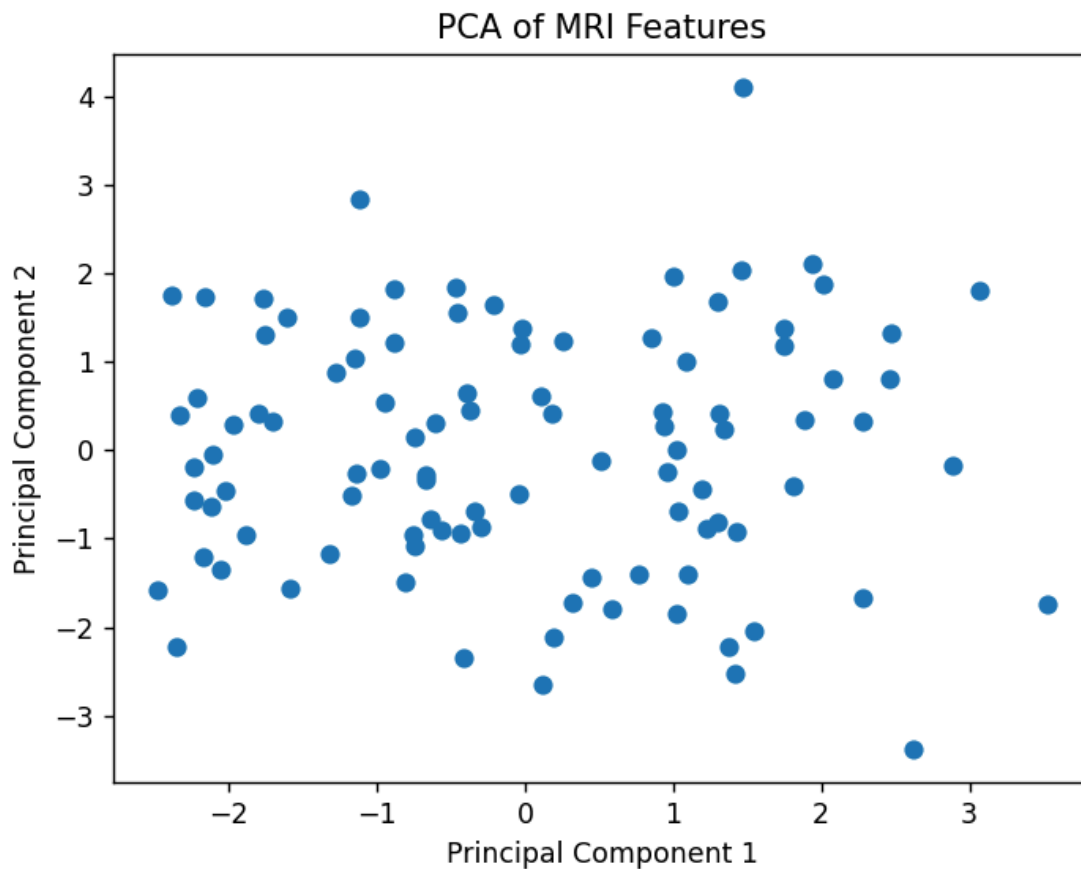
# Apply PCA
pca = PCA(n_components=2)
pca_data = pca.fit_transform(scaled_data)

# Plot transformed data
plt.scatter(pca_data[:, 0], pca_data[:, 1])

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA of MRI Features")

plt.show()

print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
```



```

-- RESTART: C:/Users/serico/AppD
Explained Variance Ratio:
[0.09073869 0.07534197]

```

Interpretation:

PCA reduces the original **25 features to 2 principal components** while retaining as much information as possible.

8. Iris Dataset — PCA, Summary and Biplot

Python

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

```

```

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Standardize
X_scaled = StandardScaler().fit_transform(X)

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Summary
print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)

print("\nTotal Variance Explained:")
print(sum(pca.explained_variance_ratio_))

# Biplot
plt.figure(figsize=(8, 6))

plt.scatter(X_pca[:, 0],
            X_pca[:, 1],
            c=y)

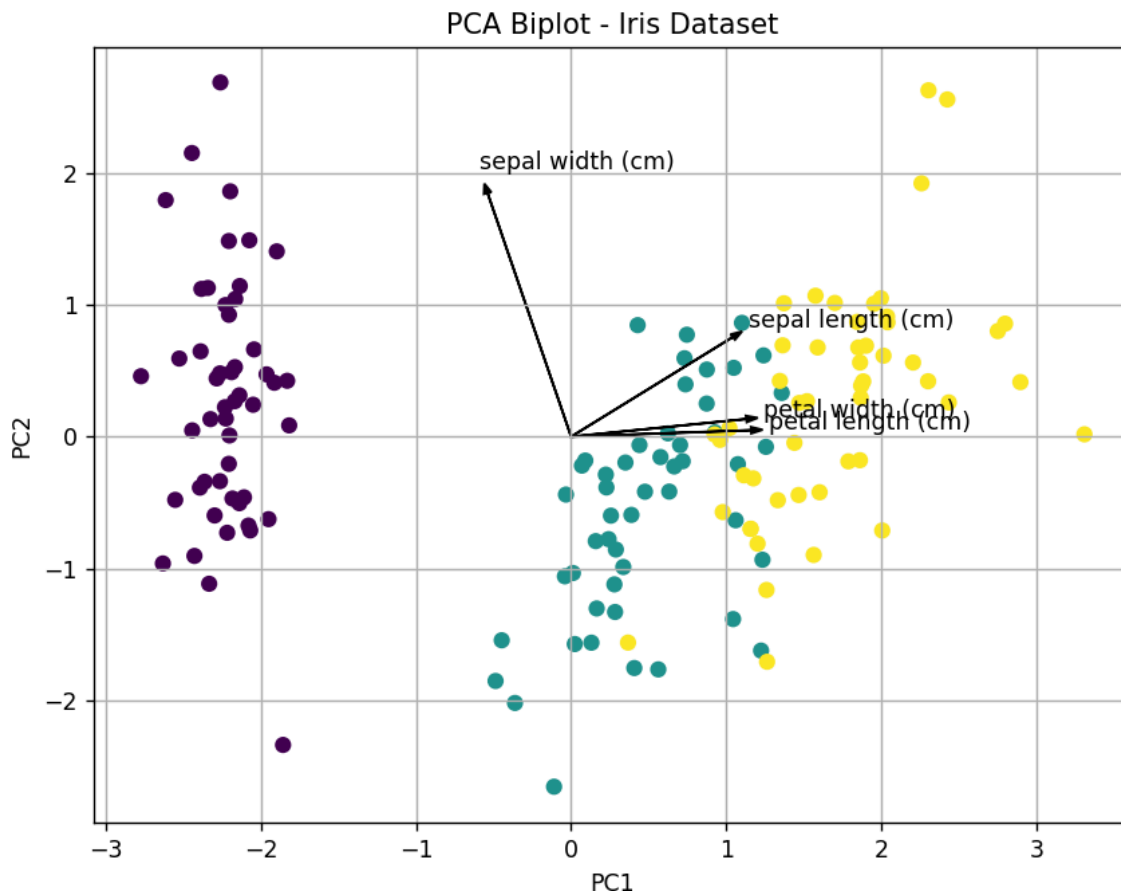
# Feature vectors
features = iris.feature_names

for i, feature in enumerate(features):
    plt.arrow(0, 0,
              pca.components_[0, i] * 2,
              pca.components_[1, i] * 2,
              color='black',
              head_width=0.05)

plt.text(pca.components_[0, i] * 2.2,
         pca.components_[1, i] * 2.2,
         feature)

```

```
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Biplot - Iris Dataset")
plt.grid(True)
plt.show()
```



```
Explained Variance Ratio:
[0.72962445 0.22850762]
```

```
Total Variance Explained:
0.9581320720000166
```

Explanation:

Principal Component 1 (PC1) explains the highest variance because it has the largest explained variance ratio. For the standardized Iris dataset, PC1 explains approximately **73% of the variance**, while PC2 explains approximately **23%**.

Together, PC1 and PC2 explain approximately **96% of the total variance**.

9. Automobile Dataset — PCA

Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Generate sample automobile dataset
np.random.seed(10)
data = np.random.rand(100, 15)

# Standardize
scaled_data = StandardScaler().fit_transform(data)

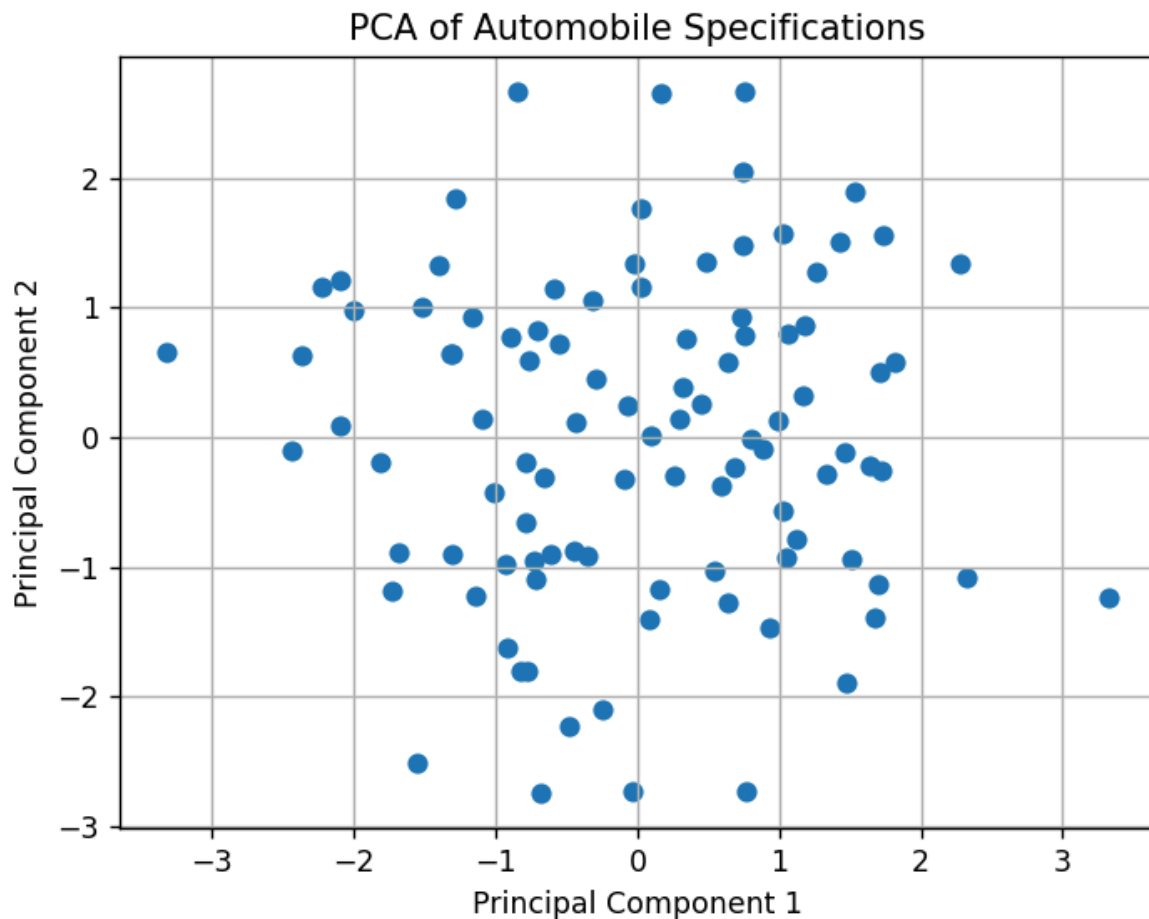
# PCA
pca = PCA(n_components=2)
pca_data = pca.fit_transform(scaled_data)

# Visualization
plt.scatter(pca_data[:, 0], pca_data[:, 1])

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA of Automobile Specifications")

plt.grid(True)
plt.show()

print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
```



Interpretation:

The original **15 automobile specifications** are reduced to two principal components. The scatterplot provides a two-dimensional representation of the automobile data.

10. Facial Recognition — t-SNE

Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler

# Generate sample facial embeddings
np.random.seed(42)
data = np.random.rand(100, 128)
```

```
# Standardize
data_scaled = StandardScaler().fit_transform(data)

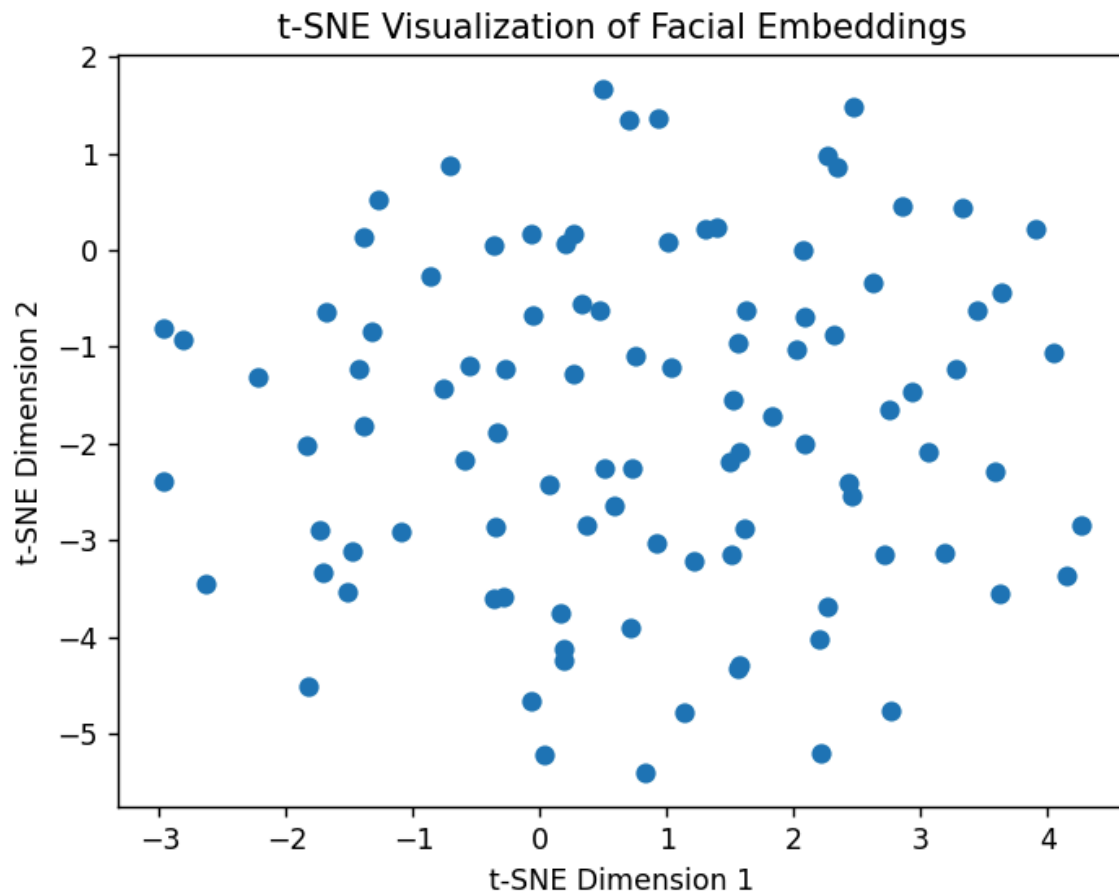
# Apply t-SNE
tsne = TSNE(n_components=2,
            random_state=42,
            perplexity=30)

data_tsne = tsne.fit_transform(data_scaled)

# Plot
plt.scatter(data_tsne[:, 0], data_tsne[:, 1])

plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.title("t-SNE Visualization of Facial Embeddings")

plt.show()
```

Why t-SNE instead of PCA?

t-SNE is preferred for visualization because it is particularly effective at preserving **local neighborhood relationships** and revealing clusters in high-dimensional data.

PCA is a **linear dimensionality reduction technique**, whereas t-SNE can capture **non-linear relationships**.

11. Iris Dataset — t-SNE

Python

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler
```

```
# Load dataset
iris = sns.load_dataset("iris")

X = iris.iloc[:, 0:4]
y = iris["species"]

# Standardize
X_scaled = StandardScaler().fit_transform(X)

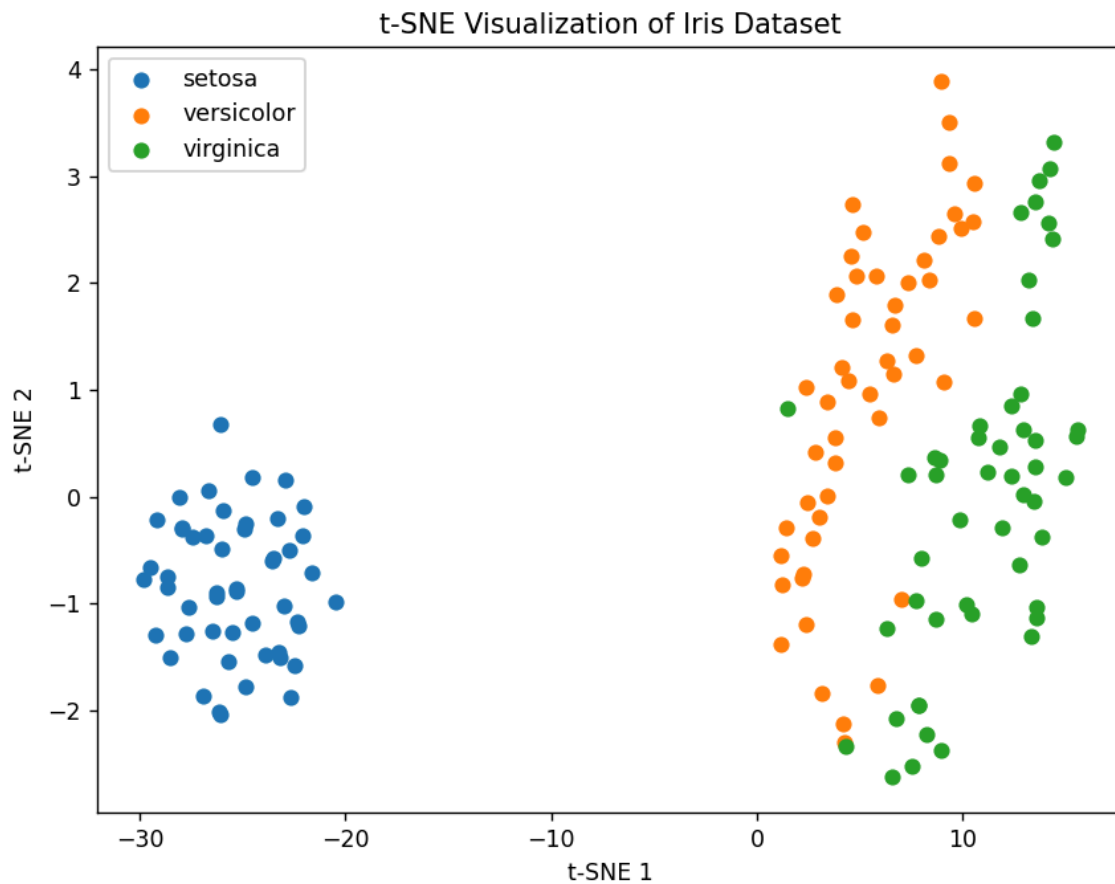
# t-SNE
tsne = TSNE(n_components=2,
            random_state=42,
            perplexity=30)

X_tsne = tsne.fit_transform(X_scaled)

# Plot
plt.figure(figsize=(8, 6))

for species in y.unique():
    index = y == species
    plt.scatter(X_tsne[index, 0],
                X_tsne[index, 1],
                label=species)

plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.title("t-SNE Visualization of Iris Dataset")
plt.legend()
plt.show()
```



Interpretation:

The t-SNE plot generally forms separate groups corresponding to the three Iris species. **Setosa** is usually clearly separated, while **versicolor** and **virginica** may show some overlap.

12. Customer Segmentation — t-SNE

Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler

# Generate sample customer data
np.random.seed(42)
data = np.random.rand(200, 40)
```

```
# Standardize
data_scaled = StandardScaler().fit_transform(data)

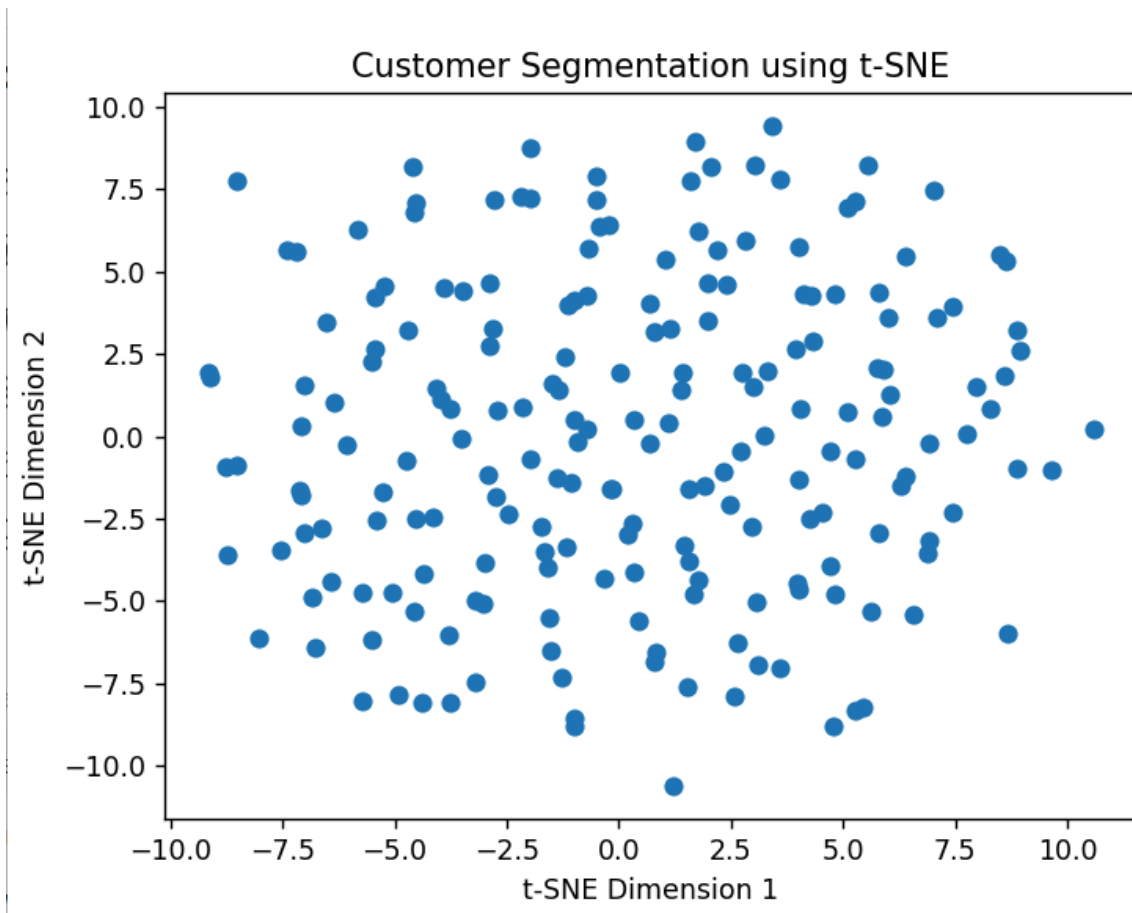
# Apply t-SNE
tsne = TSNE(n_components=2,
            random_state=42,
            perplexity=30)

result = tsne.fit_transform(data_scaled)

# Plot
plt.scatter(result[:, 0], result[:, 1])

plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.title("Customer Segmentation using t-SNE")

plt.show()
```



Discussion:

If clearly separated groups appear in the plot, distinct customer clusters exist. If the points are highly mixed, there is no strong evidence of distinct clusters. Since t-SNE is mainly a visualization technique, clustering should ideally be confirmed using methods such as **K-Means** or **DBSCAN**.

13. Medical Image Dataset — UMAP vs PCA**Python**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import umap

# Generate sample medical image data
np.random.seed(42)
data = np.random.rand(100, 100)

# Standardize
scaled_data = StandardScaler().fit_transform(data)

# UMAP
reducer = umap.UMAP(n_components=2, random_state=42)
umap_data = reducer.fit_transform(scaled_data)

# Plot UMAP
plt.scatter(umap_data[:, 0], umap_data[:, 1])

plt.xlabel("UMAP 1")
plt.ylabel("UMAP 2")
plt.title("UMAP of Medical Image Features")

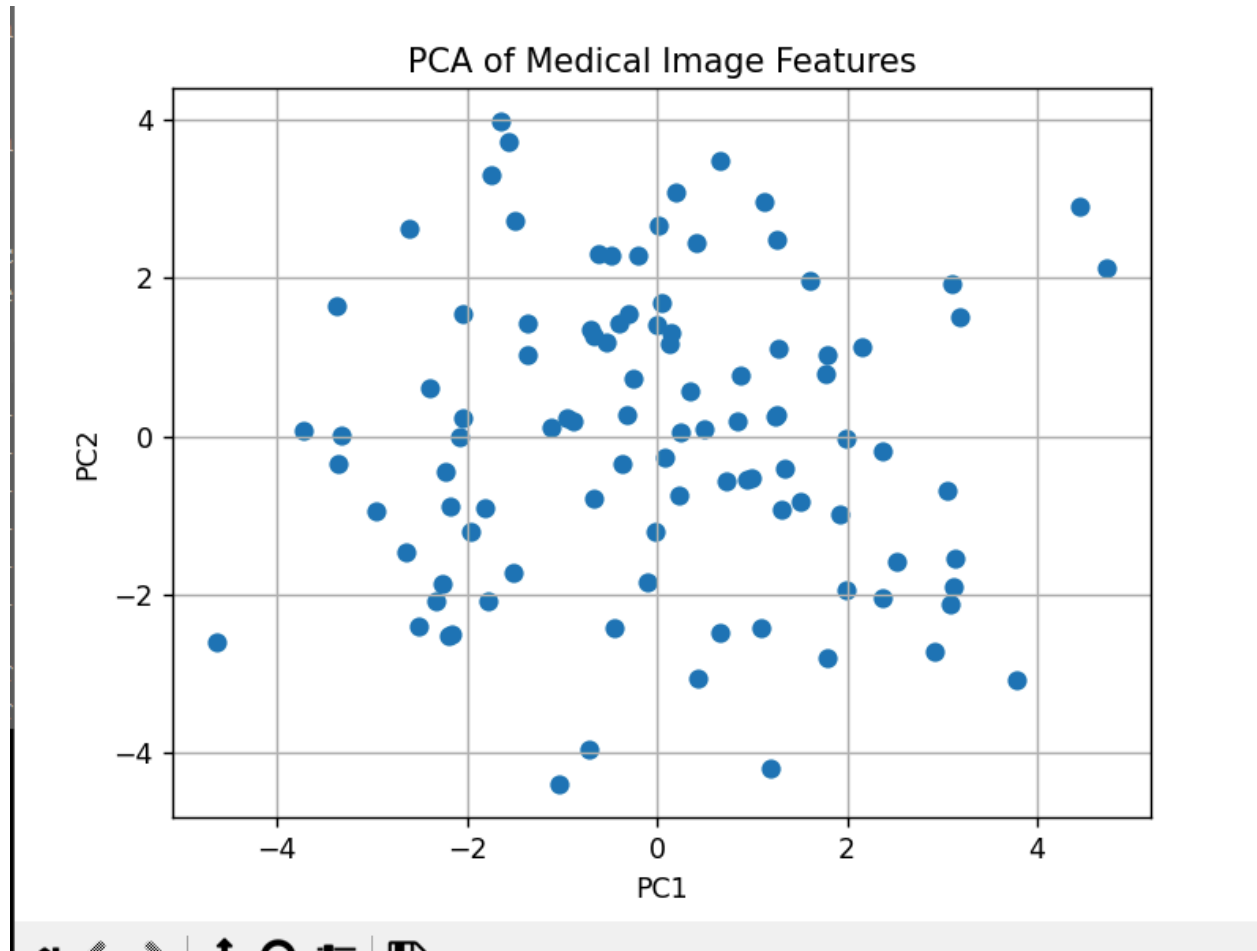
plt.show()

# PCA for comparison
pca = PCA(n_components=2)
pca_data = pca.fit_transform(scaled_data)

plt.scatter(pca_data[:, 0], pca_data[:, 1])
```

```
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA of Medical Image Features")
```

```
plt.show()
```



Comparison

PCA	UMAP
Linear technique	Non-linear technique
Preserves global variance	Preserves local structure well
Faster and simpler	Can reveal complex clusters
Good for dimensionality reduction	Good for visualization and cluster structure

Conclusion:

UMAP can reveal non-linear structures and local clusters that may not be visible using PCA.

14. Iris Dataset — UMAP**Python**

```
import seaborn as sns
import matplotlib.pyplot as plt
import umap
from sklearn.preprocessing import StandardScaler

# Load Iris dataset
iris = sns.load_dataset("iris")

X = iris.iloc[:, 0:4]
y = iris["species"]

# Standardize
X_scaled = StandardScaler().fit_transform(X)

# Apply UMAP
reducer = umap.UMAP(n_components=2,
                    random_state=42)

X_umap = reducer.fit_transform(X_scaled)

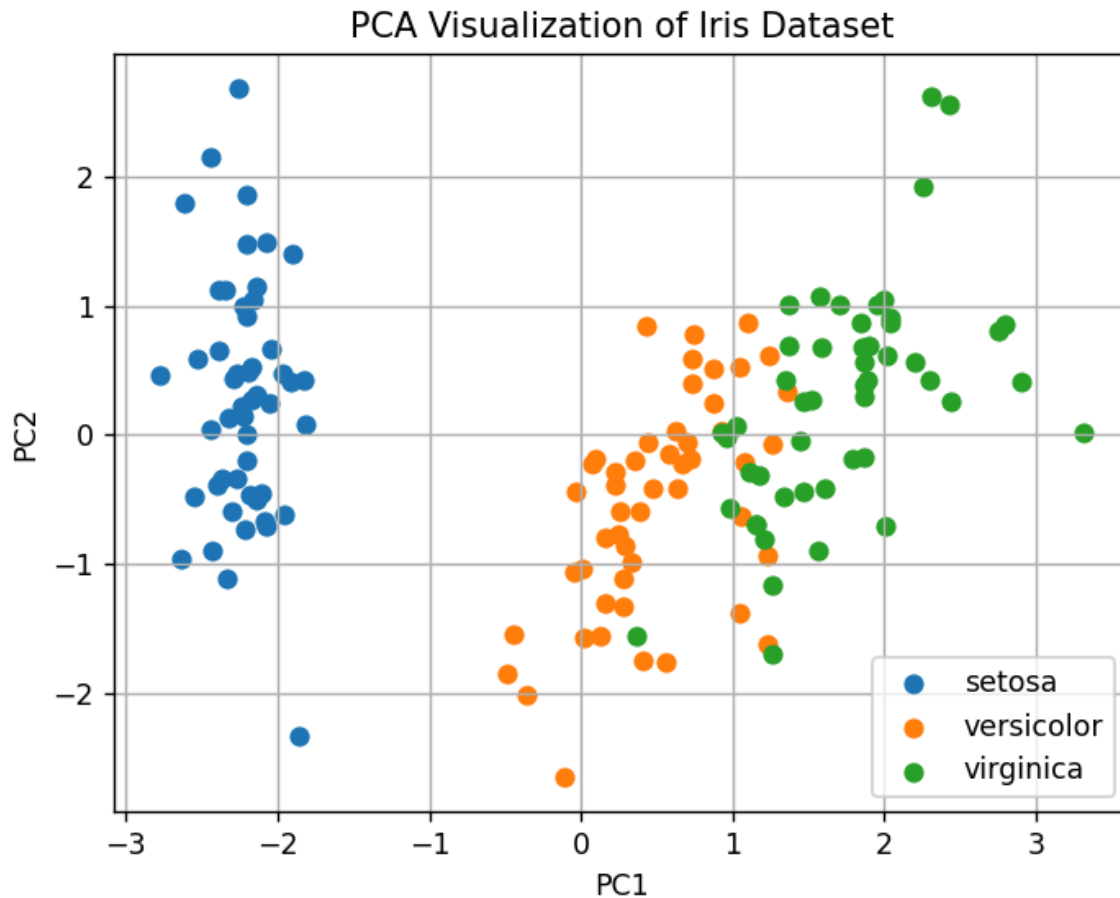
# Plot
plt.figure(figsize=(8, 6))

for species in y.unique():
    index = y == species

    plt.scatter(X_umap[index, 0],
                X_umap[index, 1],
                label=species)

plt.xlabel("UMAP 1")
plt.ylabel("UMAP 2")
plt.title("UMAP Visualization of Iris Dataset")
plt.legend()
```

```
plt.show()
```



Interpretation:

UMAP generally produces clearly separated groups for the Iris species. **Setosa** is typically well separated, while **versicolor** and **virginica** can have some overlap.

15. E-Commerce Customer Data — UMAP

Python

```
import numpy as np
import matplotlib.pyplot as plt
import umap
from sklearn.preprocessing import StandardScaler

# Generate sample e-commerce data
np.random.seed(42)
```



```
data = np.random.rand(200, 60)

# Standardize
scaled_data = StandardScaler().fit_transform(data)

# Apply UMAP
reducer = umap.UMAP(n_components=2,
                    random_state=42)

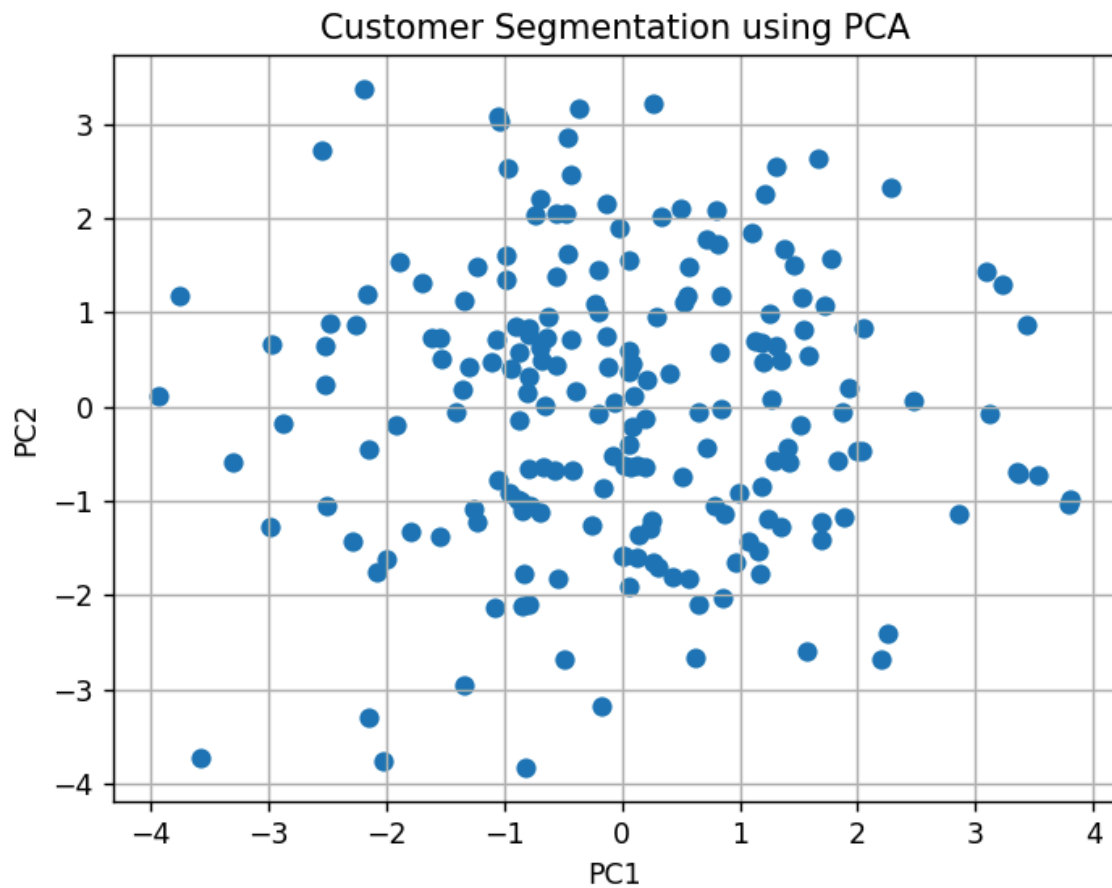
umap_result = reducer.fit_transform(scaled_data)

# Plot
plt.figure(figsize=(8, 6))

plt.scatter(umap_result[:, 0],
            umap_result[:, 1])

plt.xlabel("UMAP Dimension 1")
plt.ylabel("UMAP Dimension 2")
plt.title("Customer Clusters using UMAP")

plt.grid(True)
plt.show()
```



Interpretation:

The UMAP visualization represents the **60-dimensional customer purchasing behavior in two dimensions**. Customers that appear close together have similar behavioral characteristics. Clearly separated groups indicate potential **customer clusters or segments**.